



Rapport de Projet

Recherche opérationnelle – flots dans les graphes

Hugo CLAVEILLE

Master 1 Informatique

9 mai 2026

Table des matières

1	Introduction	2
2	Structures de données	3
3	Algorithmes de flots	4
3.1	Flot maximum	4
3.2	Flot de coût minimum	4
3.3	Cycles négatifs	5

1. Introduction

Ce projet porte sur des problèmes classiques de flots dans les graphes : le flot maximum, le flot de coût minimum et la détection de cycles négatifs. L'objectif était de construire les algorithmes nous-mêmes, sans utiliser de librairie spécialisée de type NetworkX, afin de mieux comprendre le rôle du graphe résiduel et la manière dont les solutions évoluent au fil des augmentations.

Le dépôt est organisé autour d'une structure de graphe commune, utilisée ensuite par les différents algorithmes. Cette organisation m'a paru importante : au lieu de coder chaque méthode avec son propre format de données, tous les traitements manipulent les mêmes objets `Graph` et `Edge`. Cela rend les tests plus simples, mais surtout cela met en évidence que les algorithmes de flot reposent tous sur la même idée centrale : modifier progressivement les capacités résiduelles des arcs.

Les principaux fichiers du projet sont les suivants :

- `src/graph_struct.py` : définition des structures de données et lecture des graphes depuis les fichiers texte ;
- `src/max_flow.py` : calcul d'un flot maximum avec une variante de Ford-Fulkerson basée sur un parcours en largeur ;
- `src/min_cost_flow.py` : calcul d'un flot de coût minimum par chemins augmentants de plus court coût ;
- `src/Dijkstra_min_cost_flow.py` : seconde version avec la même interface, utilisée dans les tests de comparaison ;
- `src/detect_negative_cycle.py` : détection de cycles négatifs par Bellman-Ford et normalisation simple des coûts ;
- `src/graph_to_dot.py` : export des graphes au format DOT pour visualiser les graphes et les graphes résiduels.

Le rapport présente donc les choix faits dans le code, les structures de données utilisées, puis les algorithmes implémentés et les résultats obtenus sur les exemples de test.

2. Structures de données

Pour représenter les graphes, j'ai choisi de créer une classe **Graph** qui contient une source, un puits, et le nombre de sommets n , et une liste de taille n de listes d'arcs sortants où pour chaque sommet i , `graph[i]` contient une liste d'objets **Edge** représentant les arcs sortants de i .

Chaque objet **Edge** contient les informations suivantes : le sommet d'origine, de destination, la capacité, le coût et un booléen indiquant s'il s'agit d'un arc résiduel ou non.

J'ai également implémenté quelques méthodes pour faciliter la manipulation des graphes :

- **add_edge** : pour ajouter un arc au graphe, en créant automatiquement l'arc résiduel correspondant ;
- **add_arc** : pour ajouter un arc sans créer de résiduel, utilisé principalement pour les graphes de test ;
- **get_edge** : pour récupérer tous les arcs du graphe, utile pour les algorithmes qui doivent parcourir tous les arcs ;

J'ai aussi ajouté une fonction pour tester l'égalité de deux graphes, mais cela n'a en fait aucun intérêt car les solutions de flot ne sont pas forcément uniques.

3. Algorithmes de flots

3.1 Flot maximum

Pour le calcul du flot maximum, j'ai implémenté une variante de l'algorithme de Ford-Fulkerson, plus précisément Edmonds-Karp. Le principe est de chercher à chaque étape un chemin augmentant entre la source et le puits dans le graphe résiduel. Pour cela, j'utilise un parcours en largeur, ce qui permet de choisir un chemin avec peu d'arcs et d'éviter certains mauvais comportements de Ford-Fulkerson classique.

Une fois un chemin trouvé, on calcule la plus petite capacité disponible sur ce chemin. Cette valeur correspond au flot que l'on peut encore envoyer. On diminue alors les capacités des arcs directs et on augmente celles des arcs retour. L'algorithme s'arrête lorsqu'il n'existe plus de chemin augmentant entre la source et le puits. La somme des augmentations donne alors la valeur du flot maximum.

La coupe minimale est obtenue après le calcul du flot maximum. On effectue un dernier parcours dans le graphe résiduel depuis la source : les sommets encore atteignables forment un premier ensemble, et les autres sommets forment le second. Les arcs qui vont du premier ensemble vers le second correspondent à une coupe de capacité minimale.

3.2 Flot de coût minimum

Pour le flot de coût minimum, l'idée reste proche : on envoie progressivement du flot dans le graphe résiduel, mais on choisit cette fois un chemin de coût minimal. Dans ma première version, ce chemin est trouvé avec Bellman-Ford, ce qui permet de gérer des coûts négatifs dans le graphe résiduel.

À chaque augmentation, le coût total est mis à jour en ajoutant le produit entre le flot envoyé et le coût du chemin choisi. L'algorithme continue tant qu'il existe un chemin de la source vers le puits avec une capacité résiduelle positive.

Dans une seconde version, j'ai utilisé Dijkstra pour trouver les chemins de coût minimum. Cependant, Dijkstra ne gère pas les coûts négatifs, donc j'ai dû appliquer une technique de normalisation des coûts : après chaque augmentation, les coûts sont ajustés

pour garantir qu'ils restent non négatifs.

3.3 Cycles négatifs

La détection de cycles négatifs est aussi basée sur Bellman-Ford. Après les relaxations classiques, si une arête peut encore améliorer une distance, alors le graphe contient un cycle de coût négatif.

Pour la normalisation des coûts, j'ai choisi de soustraire le coût minimum de tous les arcs sortants de chaque sommet. Cela garantit que tous les coûts deviennent non négatifs.